Title: Implementation of Deadlock Avoidance using Banker's Algorithm

Course Outcome (CO)

```
#include <stdio.h>

#define MAX 10

int main() {

int n, m;

int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];

int total[MAX], available[MAX];

int finish[MAX] = {0}, safeSeq[MAX];

printf("Enter number of processes: ");

scanf("%d", &n);

printf("Enter number of resource types: ");

scanf("%d", &m);

// Allocation matrix

printf("\nEnter Allocation Matrix:\n");

for(int i = 0; i < n; i++) {

for(int j = 0; j < m; j++) {

scanf("%d", &alloc[i][j]);

}

}

// Max matrix

printf("\nEnter Max Matrix:\n");

for(int i = 0; i < n; i++) {

for(int j = 0; j < m; j++) {
```

```
scanf("%d", &max[i][j]);
}

}
// Total resources
printf("\nEnter Total Resources:\n");
for(int j = 0; j < m; j++) {
scanf("%d", &total[j]);
}
// Need = Max - Allocation
for(int i = 0; i < n; i++) {
for(int j = 0; j < m; j++) {
need[i][j] = max[i][j] - alloc[i][j];
}
}
// Available = Total - sum(Allocation)
for(int j = 0; j < m; j++) {
int sum = 0;
for(int i = 0; i < n; i++) {
sum += alloc[i][j];
}
available[j] = total[j] - sum;
}
// Show Available
printf("\nAvailable Resources:\n");
```

```
for(int j = 0; j < m; j++) {
printf("%d ", available[j]);
}
printf("\n");
// Safety Algorithm
int count = 0;
while(count < n) {
int found = 0;
for(int i = 0; i < n; i++) {
if(!finish[i]) {
int j;
//if need <= available for all resources,process can be executed
for(j = 0; j < m; j++) {
if(need[i][j] > available[j])
break;
}

if(j == m) {
for(int k = 0; k < m; k++) {
available[k] += alloc[i][k];
}
safeSeq[count++] = i;
finish[i] = 1;
found = 1;
}
```

```
}

}

if(!found) {

printf("\nSystem is NOT in safe state.\n");

return 0;

}

}

//Safe sequence

printf("\nSystem is in SAFE state.\nSafe sequence: ");

for(int i = 0; i < n; i++) {

printf("P%d ", safeSeq[i]);

}

printf("\n");

return 0;

}
```

Objectives of the Experiment

- Explain the concept of safe vs. unsafe system states
- Implement and analyze Banker's Algorithm logic
- Identify a safe execution order of processes

1. Description of Required Matrices

Allocation Matrix

This matrix indicates how many instances of each resource type are already assigned to each process at a given moment.

Maximum Matrix

This shows the upper limit of resources each process may request during its lifetime.

Available Matrix

This represents the currently unallocated (free) resources in the system.

Need Matrix

The Need matrix is derived from the relation:
Need = Maximum − Allocation

It reflects the remaining resources each process still requires to complete execution.

2. Initial Data Setup

Allocation

P0 → 0 1 0
P1 → 2 0 0
P2 → 3 0 2
P3 → 2 1 1
P4 → 0 0 2

Maximum

P0 → 7 5 3
P1 → 3 2 2
P2 → 9 0 2
P3 → 2 2 2
P4 → 4 3 3

Available Resources

Available = [3, 3, 2]

3. Computation of Need Matrix

By subtracting Allocation from Maximum:

P0 → 7 4 3
P1 → 1 2 2
P2 → 6 0 0
P3 → 0 1 1
P4 → 4 3 1

4. Step-by-Step Execution of Safety Algorithm

Initial Available = [3, 3, 2]

Iteration 1:
Process P1 satisfies the condition (Need ≤ Available)
New Available = [5, 3, 2]

Iteration 2:
Process P3 executes
New Available = [7, 4, 3]

Iteration 3:
Process P4 executes
New Available = [7, 4, 5]

Iteration 4:
Process P0 executes
New Available = [7, 5, 5]

Iteration 5:
Process P2 executes
New Available = [10, 5, 7]

5. Safe Sequence Determination

One valid safe sequence is:
P1 → P3 → P4 → P0 → P2

Therefore, the system remains in a SAFE condition.

6. Testing an Unsafe Scenario

If the Available resources are reduced to:
Available = [0, 0, 0]

Then no process meets the requirement (Need ≤ Available).

As a result, no execution can proceed, indicating an unsafe condition.

Output:
System is NOT in safe state (Deadlock may occur)

7. Explanation

Safe Case

All processes can complete in some order without waiting indefinitely, which confirms system safety.

Unsafe Case

Since no process can start execution, the system may enter a deadlock situation.

8. Final Remarks

Banker's Algorithm is a preventive approach used in operating systems to ensure that resource allocation does not push the system into an unsafe state. By simulating allocation before actually granting it, the algorithm guarantees that a safe execution order always exists.

9. Key Observation (Additional Insight)

Even if a system is not currently deadlocked, it can still be unsafe. Banker's Algorithm helps identify such risky states in advance and avoids them proactively.